

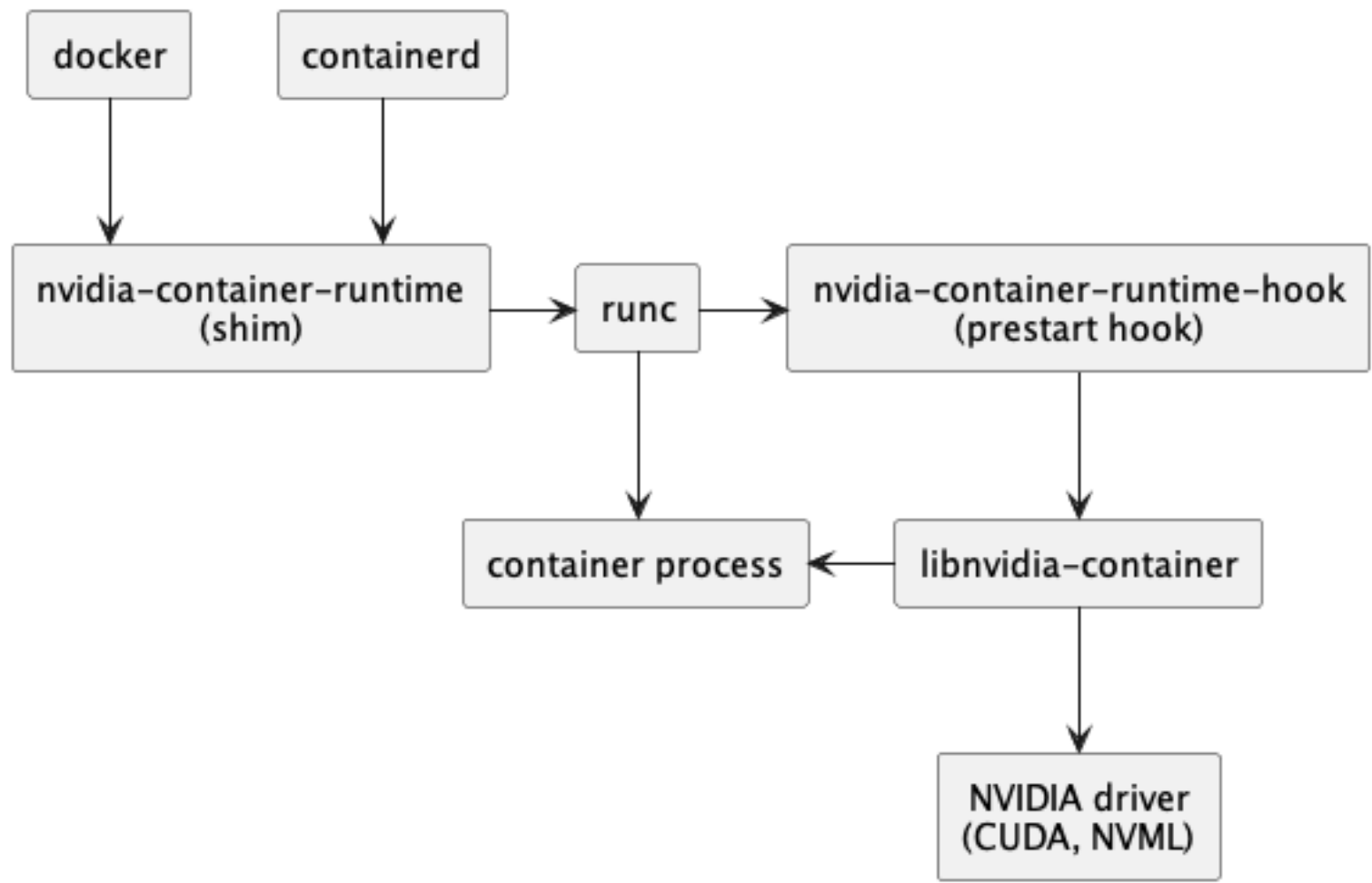
Architecture Overview

The NVIDIA container stack is designed to support any container runtime in the ecosystem. The components of the stack include:

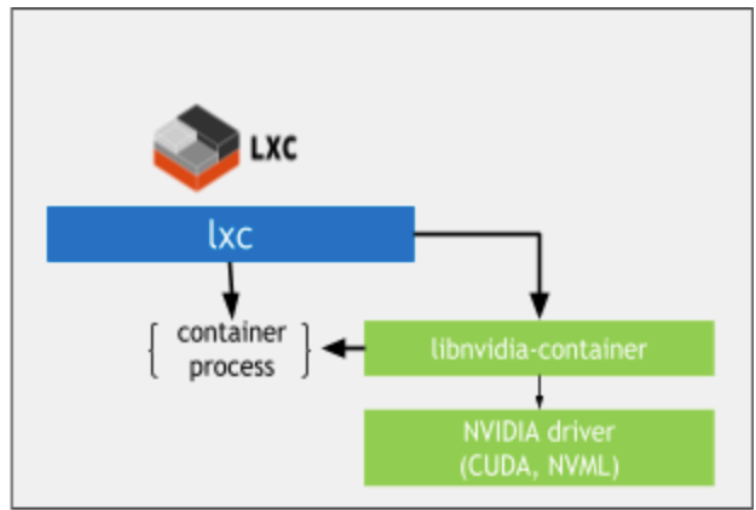
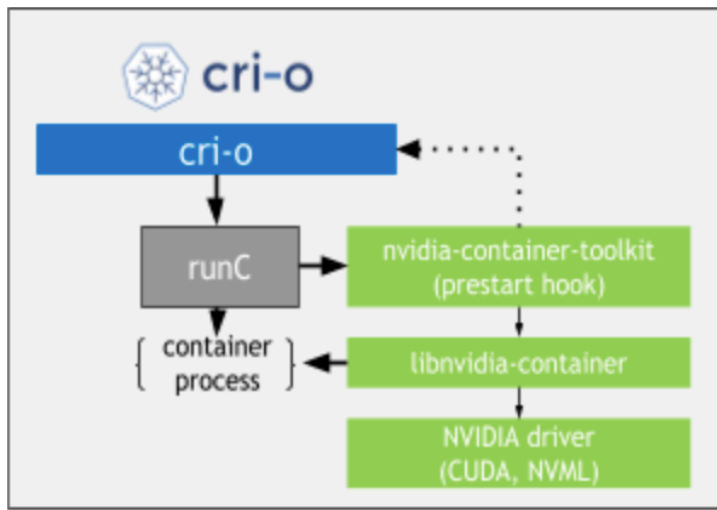
- The NVIDIA Container Runtime (`nvidia-container-runtime`)
- The NVIDIA Container Runtime Hook (`nvidia-container-toolkit` / `nvidia-container-runtime-hook`)
- The NVIDIA Container Library and CLI (`libnvidia-container1` , `nvidia-container-cli`)

The components of the NVIDIA container stack are packaged as the NVIDIA Container Toolkit.

How these components are used depends on the container runtime being used. For `docker` or `containerd`, the NVIDIA Container Runtime (`nvidia-container-runtime`) is configured as an OCI-compliant runtime. The flow through the components is shown in the following diagram:



The flow through components for `cri-o` and `lxc` is shown in the following diagram. In this case, the NVIDIA Container Runtime component is not required.



The following sections describe each component in the NVIDIA container stack, starting with the lowest-level component and working up.

Components and Packages

The main packages of the NVIDIA Container Toolkit are:

- `nvidia-container-toolkit`
- `nvidia-container-toolkit-base`
- `libnvidia-container-tools`
- `libnvidia-container1`

With the dependencies between these packages shown below:

```

├─ nvidia-container-toolkit (version)
│   └─ libnvidia-container-tools (>= version)
│       └─ nvidia-container-toolkit-base (version)
├─ libnvidia-container-tools (version)
│   └─ libnvidia-container1 (>= version)
└─ libnvidia-container1 (version)
  
```

where `version` represents the NVIDIA Container Toolkit version.

Note

In the past, the `nvidia-docker2` and `nvidia-container-runtime` packages were also discussed as part of the NVIDIA container stack. These **packages** should be considered deprecated as their functionality has been merged with the `nvidia-container-toolkit` package. The packages might still be available to introduce dependencies on `nvidia-container-toolkit` and ensure that older workflows continue to function. For more information on these packages, refer to the documentation archive for versions older than `v1.12.0`.

The NVIDIA Container Library and CLI

These components are packaged as the `libnvidia-container-tools` and `libnvidia-container1` packages, respectively.

These components provide a library and a CLI utility to automatically configure GNU/Linux containers that use NVIDIA GPUs. The implementation relies on kernel primitives and is designed to be agnostic of the container runtime.

`libnvidia-container` provides a well-defined API and a wrapper CLI (called `nvidia-container-cli`) that different runtimes can invoke to inject NVIDIA GPU support into their containers.

The NVIDIA Container Runtime Hook

This component is included in the `nvidia-container-toolkit` package.

This component includes an executable that implements the interface required by a `runC` `prestart` hook. `runC` invokes this script after a container is created but before it starts, and gives it access to the `config.json` associated with the container (such as this [config.json](#)). It then takes information contained in the `config.json` and uses it to invoke the `nvidia-container-cli` CLI with an appropriate set of flags. One of the most important flags is which specific GPU devices to inject into the container.

The NVIDIA Container Runtime

This component is included in the `nvidia-container-toolkit-base` package.

This component used to be a complete fork of `runC` with NVIDIA-specific code injected into it. Since 2019, it is a thin wrapper around the native `runC` installed on the host system.

`nvidia-container-runtime` takes a `runC` spec as input, injects the NVIDIA Container Runtime Hook as a `prestart` hook into it, and then calls out to the native `runC`, passing it the modified `runC` spec with that hook set. For versions of the NVIDIA Container Runtime from `v1.12.0`, this runtime also performs additional modifications to the OCI runtime spec to inject specific devices and mounts not handled by the NVIDIA Container CLI.

This component is not necessarily specific to Docker, but it is specific to `runC`.

The NVIDIA Container Toolkit CLI

This component is included in the `nvidia-container-toolkit-base` package.

This component is a CLI that includes several utilities for interacting with the NVIDIA Container Toolkit. This functionality includes configuring runtimes such as `docker` for use with the NVIDIA

Which Package to Use

Installing the `nvidia-container-toolkit` package is sufficient for all use cases. This package is continuously enhanced with functionality and tools that simplify working with containers and NVIDIA devices.

To use Kubernetes with Docker, you need to configure the Docker `daemon.json` to include a reference to the NVIDIA Container Runtime and set this runtime as the default. The NVIDIA Container Toolkit contains a utility to update this file as highlighted in the `docker`-specific installation instructions.

Refer to the [Installing the NVIDIA Container Toolkit](#) for more information on installing the NVIDIA Container Toolkit on various Linux distributions.

Package Repository

The packages for the components listed above are available in the `gh-pages` branch of the GitHub repos of these projects. This is useful for air-gapped deployments that need access to the packages (`.deb` and `.rpm`) to support offline installs.

For the different components:

1. `nvidia-container-toolkit`
 - <https://github.com/NVIDIA/libnvidia-container/tree/gh-pages/>
2. `libnvidia-container`
 - <https://github.com/NVIDIA/libnvidia-container/tree/gh-pages/>

☐ Note

As of the release of version `1.6.0` of the NVIDIA Container Toolkit, the packages for all components are published to the `libnvidia-container` repository [<https://nvidia.github.io/libnvidia-container/>](https://nvidia.github.io/libnvidia-container/) listed above. For older package versions, refer to the documentation archives.

Releases of the software are also hosted on the `experimental` branch of the repository and are



